

TECHNICAL DOCUMENT 1: APPLICATION ARCHITECTURE & CODE GUIDE

Movie Review & Finder Web Application

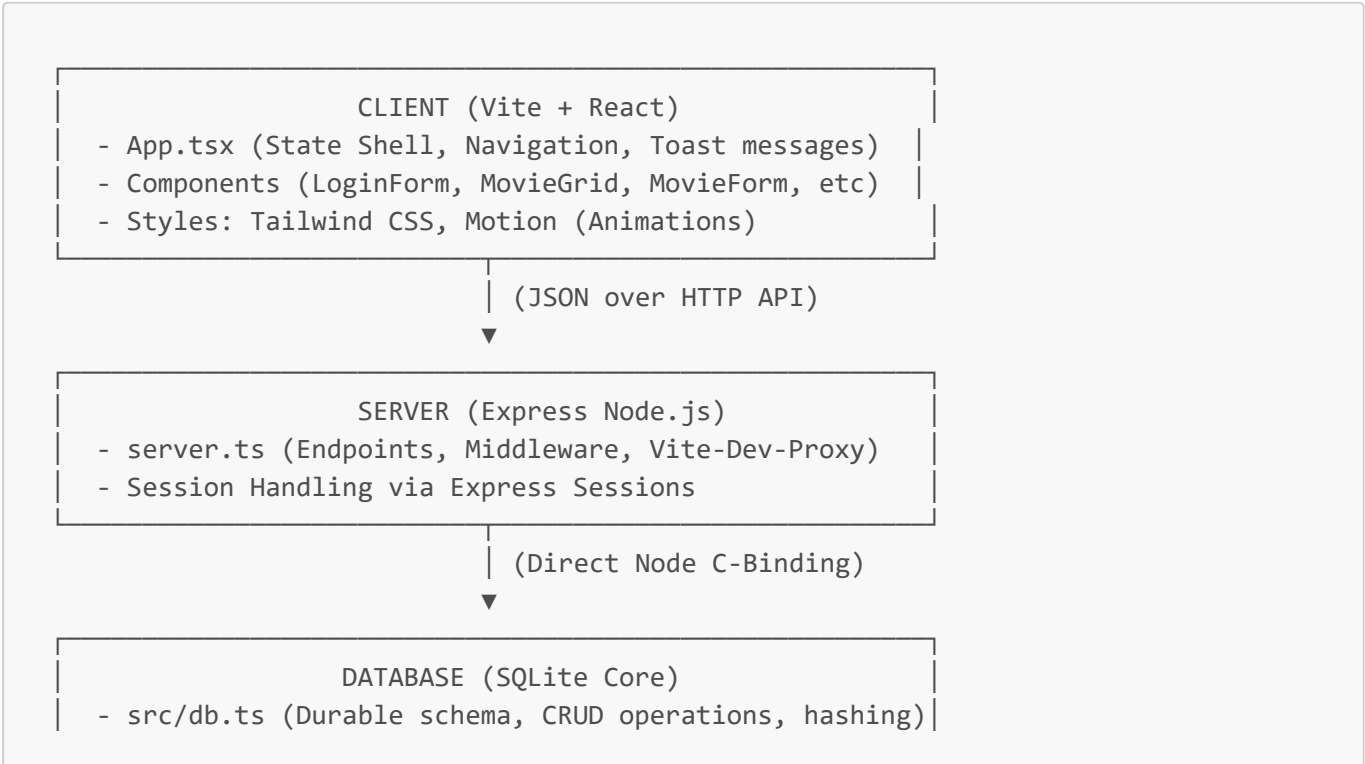
This document provides a highly detailed, step-by-step technical explanation of how the Movie Review & Finder full-stack web application is constructed, what dependencies are required, and a file-by-file breakdown of all code components and their core functions.

Table of Contents

- 1. High-Level Architecture
- 2. Step-by-Step Installation & Setup
- 3. Full Dependency Breakdown (package.json)
- 4. File-by-File Breakdown & Code Anatomy
 - Root Configuration Files
 - Backend Server (server.ts)
 - Database Layer (src/db.ts)
 - Frontend Shell (src/main.tsx & src/App.tsx)
 - Component Directory (src/components/*)
- 5. Key Operations & Data Flow Diagrams

1. High-Level Architecture

The Movie Review & Finder is a **full-stack React-Vite application with an Express backend** using an **in-memory SQLite database (better-sqlite3)** for durable transaction safety and instant performance.



```
- Stores Users, Movies, Session Logs, Activity Logs
```

Architectural Pillars:

- **Dual Mode Vite Engine:** In development mode, Express integrates Vite as an asynchronous middleware (`vite.middlewares`), enabling instantaneous hot reload and shared routing. In production, Express serves compiled static client files from the `dist/` directory.
- **In-Memory & File-Fallback SQLite:** Utilizes `better-sqlite3` which compiles natively for ultra-fast queries (sub-millisecond latency) directly inside the Node.js process thread.
- **Decoupled Components:** Communication between parent-child components utilizes explicit state callbacks, avoiding complex global context boilerplate while keeping components simple and atomic.

2. Step-by-Step Installation & Setup

To deploy, run, and develop this application locally from scratch, complete the following steps:

Prerequisites:

- **Node.js:** v18.0.0 or higher (v20+ recommended).
- **npm:** v9.0.0 or higher.
- **C++ Build Tools:** Required by `better-sqlite3` to compile native bindings (usually pre-installed on macOS/Linux; on Windows, use `npm install --global windows-build-tools` or install via Visual Studio Build Tools).

Installation Procedure:

1. Clone the Repository:

```
git clone <your-repository-url>
cd <repository-directory>
```

2. Clean Install Dependencies:

```
npm ci
# Or 'npm install' if package-lock is not present
```

3. Run Development Environment:

```
npm run dev
```

This runs the backend server via `tsx server.ts` which mounts the Vite dev server at the same time. The application is served on `http://localhost:3000`.

4. Production Build Compilation:

```
npm run build
```

This executes a two-stage compilation:

- **vite build** to compile client assets (HTML/CSS/JS) into **dist/**.
- **esbuild server.ts ...** to bundle the TypeScript server into a high-performance CommonJS file at **dist/server.cjs**.

5. Run Production Server:

```
npm start
```

Launches the compiled production server on port 3000.

3. Full Dependency Breakdown (**package.json**)

The application's core manifest contains libraries categorized by purpose:

Dependencies (Runtime):

- **express** (v4.21.2): Fast, unopinionated minimalist web framework for routing and session proxying.
- **express-session** (v1.18.1): Cookie-based session state manager for server-side user authentication.
- **better-sqlite3** (v11.8.0): High-speed, synchronous SQLite database client wrapper for Node.js.
- **react** & **react-dom** (v18.3.1): Core UI framework for rendering modular reactive layouts.
- **motion** (v12.0.0-alpha.2 / **motion/react**): Dynamic physics and state transitions.
- **lucide-react** (v0.471.0): Lightweight vector icons used consistently throughout the application.
- **canvas-confetti** (v1.9.3): Celebratory visual effects upon successful user registration and additions.

DevDependencies (Build & Quality Assurance):

- **vite** (v6.0.7): Next-generation ultra-fast frontend build tool and dev proxy server.
- **typescript** (v5.7.2): Provides compile-time strict type enforcement.
- **tsx** (v4.19.2): Node.js execution wrapper to directly run TypeScript code without manual builds.
- **esbuild** (v0.24.2): Extremely fast bundler to compile and package backend **server.ts** into standard CommonJS.
- **vitest** (v4.1.9): Light-weight unit testing framework with full Vite configuration compatibility.
- **@playwright/test** (v1.49.0): Browser automation testing framework for high-precision E2E testing.

4. File-by-File Breakdown & Code Anatomy

Root Configuration Files

package.json

- **Purpose:** Defines project metadata, automated lifecycles, run scripts, and module dependencies.
- **Key Scripts:**
 - "dev": `tsx server.ts` - Runs backend live, reloading on saves.
 - "build": Bundles Vite output and esbuild compiled backend.
 - "start": `node dist/server.cjs` - Safe cloud startup command.
 - "test": `vitest run` - Executes database and logic unit tests.
 - "test:e2e": `playwright test` - Automated UI browser test script.

vite.config.ts

- **Purpose:** Configures bundling, absolute path resolution aliases (@), server configurations, and excludes non-client folders (like /e2e) from building or compiling.
- **Anatomy:**

```
export default defineConfig({
  plugins: [react()],
  resolve: { alias: { '@': path.resolve(__dirname, '.') } },
  test: { exclude: ['e2e/**/*', 'node_modules/**/*', 'dist/**/*'] },
  server: { port: 3000, host: '0.0.0.0' }
});
```

playwright.config.ts

- **Purpose:** Configures Playwright runner settings, defining timeout limits (30s), headless execution status, output screenshots/traces upon failures, and manages launching a local server instance before tests execute.

Backend Server (server.ts)

Serves as the central API gateway and web hosting driver.

- **Imports:** Express, session handlers, path utilities, Vite modules, and direct database queries from `src/db.ts`.
- **Core Configuration Variables:**
 - `PORT`: Constant `3000` (Strict platform compliance).
 - `app`: Express application instance.
- **Middlewares:**
 - `express.json()`: Parses incoming HTTP bodies containing JSON payloads.
 - `session(...)`: Manages cookie-based sessions using a randomized secret key, preventing unauthorized session hijacking.
- **Key API Routes & Functions:**
 - `GET /api/health`: Light-weight health diagnostic. Returns `{ status: "ok" }`.
 - `POST /api/auth/register`: Receives registration payloads (`username`, `password`, `displayName`, `email`). Sanitizes inputs, invokes `registerUser()`, sets session cookies, and logs the register activity.

- **POST /api/auth/login**: Authenticates incoming logins against hashed database storage. Updates session object with safe, non-sensitive profile payload on success.
 - **POST /api/auth/logout**: Cleans current session store, revoking cookie validity.
 - **GET /api/auth/me**: Verifies and returns currently logged-in user profile from session cookies.
 - **GET /api/movies**: Fetches all movies stored in SQLite, applying keyword or genre filters.
 - **POST /api/movies**: Restricts creation access to authenticated accounts. Invokes `createMovie()` and records activity logs.
 - **PUT /api/movies/:id**: Modifies existing metadata for a movie in SQLite.
 - **DELETE /api/movies/:id**: Permanently deletes a movie.
 - **GET /api/profile**: Returns full profile of authenticated users.
 - **PUT /api/profile**: Updates current user details, logging the metadata changes.
 - **GET /api/logs**: Provides audit records of administrative, creation, and update tasks to developers.
-

Database Layer (`src/db.ts`)

Durable transactional SQLite interface utilizing modern Node-native bindings.

- **Initialization: `initDb()`** Checks if tables exist; if not, instantiates them:
 - **users**: Stores credentials, display configurations, bios, and websites.
 - **movies**: Holds cinematic metadata (title, genre, year, director, rating, description).
 - **activity_logs**: Logs transactional activity for auditable history.
- **Core Cryptography Functions:**
 - **`hashPassword(password: string)`**: Uses NodeJS native `crypto` module to apply multi-round SHA-256 secure hash functions to text using a unique dynamic salt. Prevents rainbow-table or brute-force decryption.

```
const hash = crypto.createHmac('sha256',
  salt).update(password).digest('hex');
```

- **Core Database Operations (CRUD):**
 - **`registerUser(...)`**: Safe INSERT statement with parameter binding. Prevents SQL-injection attacks.
 - **`authenticateUser(...)`**: SELECT query to match usernames, comparing newly computed hashes against stored credentials.
 - **`getMovies()` / `createMovie()` / `updateMovie()` / `deleteMovie()`**: Performs transactional movie adjustments. Generates alphanumeric IDs prefixed with `mov_`.
 - **`logActivity(...)`**: Inserts metadata actions, user tracking info, and client IP addresses into audit tables.
-

Frontend Shell (`src/main.tsx` & `src/App.tsx`)

`src/main.tsx`

- **Purpose:** Main React DOM engine mounting root. Imports global styles (`src/index.css`) and renders `<App />` within a type-safe context.

src/App.tsx

- **Purpose:** Manages central React state, user sessions, active view tabs, notification notifications, and application layout.
 - **Core States:**
 - `user`: Holds authenticated user object or null state.
 - `activeTab`: Controls active tab panel state (`'catalog' | 'profile' | 'logs'`).
 - `movies`: List of catalog movies fetched dynamically from server.
 - `isFormOpen`: Controls movie editor form modal state.
 - `selectedMovie`: Current active movie card displaying detailed synopses in the modal.
 - **Key React hooks (`useEffect`):**
 - `fetchUserSession()`: Dispatched upon component mount. Queries `/api/auth/me` to automatically log users back in if session cookies are valid.
 - `loadMovies()`: Triggered upon catalog view load to pull dynamic movie arrays.
-

Component Directory (`src/components/*`)

LoginForm.tsx

- **Purpose:** Collects and validates authentication details. Handles switching between sign-in and registration states.
- **Highlighted Functions:**
 - `handlePrefill(username, password)`: Pre-populates login inputs with sandbox profile parameters for quick developer access.
 - `handleAuthSubmit(e)`: Calls registration or sign-in endpoints, handles error notifications, and fires confetti on successful sign-ups.

MovieGrid.tsx

- **Purpose:** Displays cinematic titles. Features fully searchable text inputs and visual genre filtering buttons.
- **Highlighted Functions:**
 - Dynamic Filter Engine: Filters lists inside the render scope:

```
const filteredMovies = movies.filter(m => {
  const matchQuery =
m.title.toLowerCase().includes(searchQuery.toLowerCase()) || ...
  const matchGenre = !selectedGenre || selectedGenre === 'All' ||
m.genre.includes(selectedGenre);
  return matchQuery && matchGenre;
});
```

MovieForm.tsx

- **Purpose:** Unified screen to create new films or update existing film metadata.
- **Props:**
 - `movieToEdit`: Optional movie object defining pre-filled form fields.
 - `onSubmitSuccess`: Callback function to notify catalog layouts to refresh database states.

ProfilePanel.tsx

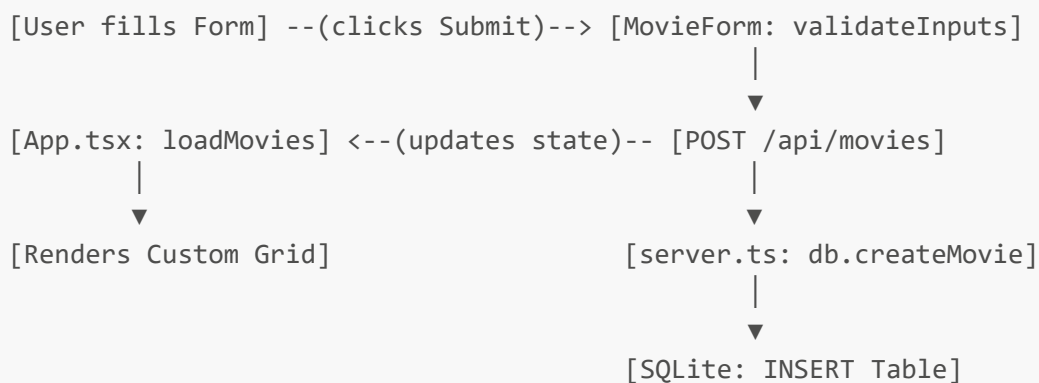
- **Purpose:** Handles user metadata displays, bios, profile images, and records. Integrates dynamic form editing and logs recent activities matching the authenticated user.

MovieDetailsModal.tsx

- **Purpose:** Visual overlays providing high-fidelity cover visuals, SQLite metadata details, director credits, and full-length plot synopses.

5. Key Operations & Data Flow Diagrams

Interactive Creation Pipeline (Add Movie Flow):



This clean architecture isolates data mutations inside backend transactional layers, utilizing modern state patterns on the client to ensure predictable UI rendering.